

ECE 473/573  
Cloud Computing and Cloud Native Systems  
Lecture 02 Cloud Computing and  
Cloud Native Systems

Professor Jia Wang  
School of Engineering, Applied Science, and Health  
Illinois Institute of Technology

August 19, 2026

# Outline

Cloud Computing

Cloud Native Systems

# Reading Assignment

- ▶ This lecture: 1, 6
- ▶ Next lecture: 2, 3

# Outline

Cloud Computing

Cloud Native Systems

# Meeting Personal Computing Needs

- ▶ Buy a desktop or laptop computer that meets the need.
  - ▶ Or, source parts and assemble a computer by yourself.
- ▶ Things you have taken for granted.
  - ▶ A place to use the computer.
  - ▶ Power that is readily available.
  - ▶ Internet connection that is also available.

# Meeting Business Computing Needs

- ▶ Needs are more diverse than those for personal computing.
- ▶ Demand of computing power beyond a single computer.
  - ▶ Demand may vary, e.g. peak vs off-peak times.
- ▶ Facility to hold many computer servers physically.
  - ▶ Space and cooling.
  - ▶ Power delivery.
  - ▶ Network bandwidth.
- ▶ Redundancy in case of power and network failures.
- ▶ Have multiple facilities across the world.
  - ▶ Better performance to serve users locally.
  - ▶ Better redundancy in case of extreme events.
- ▶ Will existing software running on a single computer adapt automatically with multiple servers?

# Rise of Cloud Computing

- ▶ Supercomputers for high-performance computing (HPC)
  - ▶ Primarily target at R&D of national interest that requires substantial computational power.
  - ▶ Specially made computers that are highly-reliable.
  - ▶ Need special skills to develop HPC software.
  - ▶ Very high up-front cost, difficult to expand.
- ▶ 2004: Google MapReduce
  - ▶ A programming model and framework to simplify software development for parallel data processing.
  - ▶ Run on cluster of commodity computer servers.
    - ▶ Automatically handle server failures.
    - ▶ Allow to expand cluster with more servers as business grows.
- ▶ 2006: Amazon Web Services
  - ▶ Including Amazon S3 for storage and EC2 for computing.
  - ▶ Allow businesses to rent computing resources on demand.
    - ▶ Without high up-front cost and long time to build physical infrastructure.

# Why Cloud Computing?

- ▶ An economically viable solution for business computing needs.
- ▶ Economies of scale in cloud computing
  - ▶ Fixed costs for facilities are shared among customers.
  - ▶ Bulk purchasing power allows cloud providers to negotiate better deals with suppliers on servers etc.
  - ▶ Diverse range of customers smooth the demand of computing power over time and increase resource utilization.
  - ▶ Operation of large number of servers requires automated tools to reduce human errors, which in turn reduces labor costs.
- ▶ Open-source movement reduces cost of cloud software development
  - ▶ No software license fee
  - ▶ Encourage interoperability
  - ▶ Reduce cost to train new workforce

# Categories of Cloud Services

- ▶ Infrastructure as a Service (IaaS)
  - ▶ Developers rent virtualized computing resources like virtual machines, storage, and networking components.
  - ▶ Allow developers to have complete control over software stack including networking, OS, library, and application.
- ▶ Platform as a Service (PaaS)
  - ▶ Developers rent pre-configured tools, libraries, and services like databases and message queues.
  - ▶ Allow developers to focus on core business logic while maintenance of common software components are outsourced.

# Categories of Cloud Services (Cont.)

- ▶ Software as a Service (SaaS)
  - ▶ End-users rent applications like email that can be accessed anywhere via a browser or a dedicated client.
  - ▶ Developers manage operation and maintenance of the software application to avoid compatibility issues with end-users' hardware and software systems.
  - ▶ End-users no longer need to worry about updates and upgrades other than their own devices.

# Pricing: Subscription-based

- ▶ SaaS are usually subscription-based.
  - ▶ End-users have a clear understanding of the cost.
  - ▶ Providers have a predictable and steady revenue stream, making it easier to plan and allocate resources.
  - ▶ Perpetual licensing is not a good fit when software needs to be updated and upgraded often.
- ▶ Virtual Private Servers (VPS) provide subscription-based IaaS
  - ▶ Pre-defined packages of cores, memory, storage, and networking resources for a fixed monthly or yearly fee.
  - ▶ A good option to try cloud computing for beginners.

# Pricing: Pay-per-Use

- ▶ IaaS and PaaS are usually charged pay-per-use.
  - ▶ Cores (CPU) and memory are charged per quantity per hour when the server is running.
  - ▶ Storage are charged per quantity per hour once it is created, even if the server is not running.
  - ▶ Network traffics are charged per bytes depending on direction.
  - ▶ Database queries are charged per each depending on functions.
  - ▶ However, it may become very complicated to estimate the amount of usage and thus charge for actual applications.
- ▶ Overall, pay-per-use pricing allows to optimize usage to reduce costs while subscription-based pricing is more predictable.
  - ▶ Work on Homework 1 to understand cloud pricing better.

# Outline

Cloud Computing

Cloud Native Systems

# Networked Applications

- ▶ In early days of computing when there is only mainframes, all programs ran and all data was stored in a single location.
- ▶ With inexpensive network-connected PCs that can perform non-trivial tasks, multitiered architecture was introduced.
  - ▶ Data management tier: database server
  - ▶ Business logic tier: web and application server
  - ▶ Presentation tier: PC (and mobile devices)
  - ▶ Connected via networks and can be replaced independently.
- ▶ Complexity of software increases beyond what can be managed by a single developer team efficiently.
  - ▶ Tiers, especially business logic, are decomposed into microservices.
  - ▶ These systems are now distributed systems, which behave very differently than a single or a few interconnected computers.
- ▶ At the same time, SaaS and IaaS business practices emerge.

# Scaling

- ▶ With SaaS practices, it is possible to have exponential growth in customers.
  - ▶ No need to distribute software physically, e.g. via disks.
  - ▶ Can the services scale to meet the growth as soon as possible?
  - ▶ Can the services respond to the demand dynamically to maximize the profit?
- ▶ With IaaS offerings, especially S3 and EC2 from AWS, scaling seems economically viable.
  - ▶ No need to build facilities and buy physical servers.
- ▶ Scaling is not easy.
  - ▶ Managing those resources manually at such scale is impossible.
  - ▶ Components in a distributed system, e.g. servers and network connections, will fail.
  - ▶ Most distributed systems will have nondeterministic behaviors – it is very difficult to reproduce the same outputs even if the inputs are the same.

# What Is Cloud Native?

- ▶ A paradigm to design systems that can fully utilize the cloud computing architecture.
  - ▶ Promoted by the Cloud Native Computing Foundation.
- ▶ Cloud native attributes
  - ▶ Scalable
  - ▶ Loosely coupled
  - ▶ Resilient
  - ▶ Manageable
  - ▶ Observable

# Scalability

- ▶ The ability of a system to continue to behave as expected in the face of significant upward or downward changes in demand.
  - ▶ Not necessarily a must have for initial system design.
  - ▶ But hard to remedy for at a later time.
- ▶ Vertical scaling
  - ▶ Improve performance of an instance (server or virtual machine) by adding cores, memory, storage, etc.
  - ▶ Usually requires no software change but improvements are limited due to physical limits.
- ▶ Horizontal scaling
  - ▶ Improve performance of the system by using more instances.
  - ▶ Increased complexity in system design and management.
- ▶ If the service needs to be scaled by hundreds or thousands times, horizontal scaling is the only choice.
  - ▶ Unfortunately, not all computations can be horizontally scaled.
  - ▶ In particular computations as finite state machines whose state cannot be decomposed, e.g. counting.

# Loose Coupling

- ▶ Components in a system have minimal knowledge of any other components.
  - ▶ Thus a component can be changed without requiring to change other components.
  - ▶ E.g. web servers and web browsers.
- ▶ Components need to communicate to each other using certain standard protocols.
- ▶ Note that the coupling needs to be loose and the knowledge needs to be minimal.
  - ▶ If every component need to communicate with and thus has knowledge of many other components, the system will become a distributed monolith – a nightmare of microservices.

# Resilience

- ▶ A measure of how well a system withstands and recovers from errors and faults.
  - ▶ A resilient system continues operating correctly, possibly at a reduced level, rather than failing completely.
- ▶ Failures result from faults
  - ▶ Any system can contain defects or faults, e.g. bugs.
  - ▶ Faults can lead to errors, which can cause failures.
  - ▶ Failures can propagate from components to components, and then the whole system.
- ▶ Build resilient systems
  - ▶ Prevention of all faults is unrealistic and unproductive.
  - ▶ Should assume components will fail and limit their impacts.
  - ▶ Use mechanisms like redundancy, circuit breakers, retry logic, intentional failure.

# Manageability

- ▶ The ease of modifying system behavior to ensure security, and smooth operation, and to meet changing requirements.
  - ▶ Without altering its code.
- ▶ For example, consider a system with a service and a database.
  - ▶ How to update the reference to the database in the service?
  - ▶ What if multiple versions of services and databases need to coexist for troubleshooting and performance evaluation.
  - ▶ Make use of environment variables instead of hardcoding.
- ▶ Manageable systems adapt to changing requirements
  - ▶ Feature flags
  - ▶ Credential rotation
  - ▶ Component deployment (upgrades and downgrades)
  - ▶ Instead of ad hoc code changes

# Observability

- ▶ The ability to answer unanticipated questions quickly with minimal prior knowledge and without reinstrumentation.
  - ▶ Make it possible to troubleshoot issues without reproducing nondeterministic behaviors.
  - ▶ In particular where an issue is.
- ▶ Observability in modern distributed systems
  - ▶ Build upon metrics, logging, and tracing.
  - ▶ Collected data to infer internal states from external outputs.

# Why Cloud Native?

- ▶ Dependability: produce dependable services in unreliable environments to keep users happy.
  - ▶ Availability: the ability of a system to perform its intended function at a random moment in time, e.g. uptime
  - ▶ Reliability: the ability of a system to perform its intended function for a given time interval, e.g. MTBF
  - ▶ Maintainability: the ability of a system to undergo modifications and repairs.
- ▶ Developers can no longer just prioritize feature development and leave dependability to system administrators.
  - ▶ Fault prevention
  - ▶ Fault tolerance
  - ▶ Fault removal
  - ▶ Fault forecasting

# Fault Prevention

- ▶ Good programming practices, e.g. test-driven development
- ▶ Language features, e.g. garbage collection
- ▶ Scalability ensures correct behavior as demand changes significantly, but could be source of additional faults.
- ▶ Loose coupling reduces the risk of cascading failures as faults propagate from one component to another.

# Fault Tolerance

- ▶ Resilience in two steps: error detection and recovery
- ▶ Seemingly simple case: retry failed requests.
  - ▶ Will retrying cause more faults?
- ▶ Make use of redundancy: have multiple copies of critical components.
  - ▶ Will a majority vote serve both as mechanisms for error detection and recovery?

# Fault Removal

- ▶ Source of faults
  - ▶ Implementation: human errors in system design and development, i.e. bugs
  - ▶ Environment: unexpected inputs or operation conditions.
- ▶ Verification and testing
  - ▶ Remove faults at development time.
- ▶ Manageability
  - ▶ Adapt to the environment: adjust resource usage, remedy security issues, turn features on or off, etc.

# Fault Forecasting

- ▶ Observability helps to avoid guesswork on predicting future behavior of the system

# Summary

- ▶ Cloud computing provides an economically viable solution for business computing needs.
- ▶ Cloud native systems provide dependability on top of unreliable cloud computing environments.